Improving Indoor Localization Through Robust WiFi Ranging and Sensor Fusion

Ned Bitar, Pranav Mehta, Diya Arun

Abstract

This report presents the progress made to date on our indoor localization system combining IEEE 802.11mc Fine Time Measurement (FTM) WiFi ranging with Inertial Measurement Unit (IMU) sensor fusion via an Extended Kalman Filter (EKF). We report successful implementation of the WiFi FTM ranging pipeline on ESP32-S3 hardware, integration of an IMU sensor for dead-reckoning, and an initial EKF implementation for fusing these modalities. In parallel, we conducted preliminary sensor fusion experiments using a Madgwick filter with dual IMUs to characterize orientation estimation performance. We further present a MATLAB simulation of the full 8-state EKF under realistic walking trajectories and NLoS-corrupted range measurements, which serves as our algorithmic ground truth. We report live hardware validation of the full integrated system, demonstrated via a real-time browser-based dashboard served from the device itself.

1 Motivation

Indoor localization remains a critical open problem with applications spanning asset tracking, robotics, emergency response, and pedestrian navigation. Global Navigation Satellite Systems (GNSS) are ineffective indoors due to severe RF attenuation and multipath propagation caused by walls, furniture, and building infrastructure, which has motivated a shift toward alternative RF technologies. Existing indoor positioning systems have leveraged technologies such as Bluetooth Low Energy (BLE), Ultra-Wideband (UWB), and Wi-Fi, each offering different trade-offs between accuracy, range, infrastructure cost, and device availability [1]. Among these, WiFi is particularly attractive due to its ubiquity in modern consumer devices. Early Wi-Fi localization systems relied on received signal strength (RSS) fingerprinting, which requires extensive offline training and is sensitive to environmental changes [2]. More recent approaches utilize physical-layer measurements such as Channel State Information (CSI), Angle of Arrival (AoA), and Time of Flight (ToF) to achieve improved accuracy. Among these, ToF-based ranging has gained significant attention due to its direct relationship with propagation distance.

The IEEE 802.11mc WiFi standard introduced the Fine Time Measurement (FTM) protocol, enabling round-trip time (RTT) based ranging between a client device and an access point (AP) [3]. In principle, FTM allows meter-level ranging accuracy using commodity Wi-Fi hardware, making it a good choice for localization. However, practical deployment challenges on commercial off-the-shelf devices remain. Positioning reliability degrades in environments with sparse AP coverage where geometric diversity is poor, a condition known as Dilution of Precision (DOP). In addition, hardware limitations and scanning and processing overhead need to be considered when implementing ranging and localization on device, in order to avoid compromising the device's primary throughput or reliability for standard data communication. This project builds on the existing FTM research by adding two key contributions: (1) integration of a 6-axis IMU with an Extended Kalman Filter (EKF) for continuous dead-reckoning and sensor fusion with range estimates between WiFi

measurement epochs to address DOP, and (2) low-level firmware optimizations to reduce the computational overhead of ranging and localization. The system targets low-cost ESP32-S3 hardware for both ranging anchors and the mobile node.

2 Background

Practical implementations of FTM ranging on commercial off-the-shelf devices often exhibit ranging errors on the order of several meters. These inaccuracies primarily arise from hardware timestamping limitations, clock offsets, and multipath propagation, particularly in non-line-of-sight (NLOS) indoor environments [4]. Several works have proposed calibration and statistical correction techniques to mitigate FTM ranging errors. Systems such as FUSIC rely on packet averaging and empirical bias correction to improve ranging accuracy [5]. While effective in static scenarios, these approaches require multiple packet exchanges, reducing temporal resolution and limiting applicability to mobile targets.

Recent research has demonstrated that multipath-induced errors can be addressed by explicitly analyzing and correcting distortions in the Channel Impulse Response (CIR). Wi-PRO (Wireless Positioning using Range Offsets) introduces a physical-layer approach that recovers the true line-of-sight (LOS) arrival time by reconstructing the CIR and performing bandwidth-compensated leading-edge detection [6]. By recovering nulled subcarriers through convex optimization and compensating for bandwidth-induced bias, Wi-PRO achieves meter-level ranging accuracy using a single FTM exchange on low-cost ESP32 hardware, significantly outperforming prior FTM-based systems. For this reason, we chose to base our system around WiPro [6] and add enhancements to the existing WiPro implementation. We summarize the Wi-Fi sensing model that provides the technical foundation underlying our system in the next couple sections.

2.1 Two-Way Ranging and FTM

The IEEE 802.11mc Fine Time Measurement (FTM) protocol enables range estimation between two Wi-Fi radios—an *initiator* and a *responder*—by measuring the times at which exchanged packets are transmitted and received. Denoting $t^{(0)}$ as the time of the responder's transmission to the initiator, $t^{(1)}$ as the initiator's reception time, and $t^{(2)}, t^{(3)}$ as the corresponding TX/RX times for the reverse packet, the propagation delay τ and clock offset Δ_t are given by:

$$\tau = \frac{1}{2}\left[(t^{(3)} - t^{(2)}) + (t^{(1)} - t^{(0)})\right] \tag{1}$$

$$\Delta_t = \frac{1}{2}\left[(t^{(3)} - t^{(2)}) - (t^{(1)} - t^{(0)})\right] \tag{2}$$

In free space or a single-path channel, τ directly yields the distance $d = c\tau$. However, in indoor environments, multiple propagation paths of different delays τ_i and attenuations α_i are present, and the wireless channel is more accurately modeled as:

$$h(t) = \phi_{tx-rx} \sum_{i=1}^{P} \alpha_i \, \delta(t - \tau_i) \tag{3}$$

where P is the number of paths and ϕ_{tx-rx} is a constant phase offset due to transmitter-receiver local oscillator mismatch. In this multipath setting, the Wi-Fi radio places its reception timestamp near the centroid of the channel impulse response (CIR) rather than at the true line-of-sight (LOS) arrival time, causing τ in equation (1) to systematically overestimate the true range.

A classical remedy is *First-Peak-Index* detection: find the first peak in $|h(t)|$ above a threshold. This works when all paths are widely separated in delay, but fails when nearby paths merge into a single peak. Wi-PRO instead uses a *compensated leading-edge detection* strategy adapted from UWB literature. The channel first rises above threshold T at time $\inf_t\{|h(t)| > T\}$, but in a low-bandwidth system like Wi-Fi each path is convolved with a sinc function, causing the leading edge to arrive earlier than the true path delay. Wi-PRO compensates for this by computing the expected sinc lead time t_{sinc} analytically, yielding a corrected channel rise time:

$$t_{\text{rise}} = t_{\text{sinc}} + \inf_t \{|h(t)| > T\} \tag{4}$$

This correction is then applied symmetrically to both timestamps of the FTM exchange. Assuming the uplink and downlink CIRs are identical ($h_u(t) = h_d(t)$), the bias-corrected propagation delay becomes:

$$\tau_{\text{LOS}} = \tau + t_{\text{rise}}^{(1)} \tag{5}$$

yielding a range estimate that is aligned with the true LOS path rather than the centroid of the multipath cluster.

2.2 Limitations and Motivation for IMU Fusion

Wi-PRO achieves 3–4× better median ranging accuracy than standard FTM on commodity hardware, with a median localization error of 1.5 m over a 1440 m^2 office building. However, the paper itself notes that performance degrades in sparse-anchor environments and under severe NLoS, and that fusing range estimates with inertial sensors would further improve accuracy [6]. In environments where the geometric diversity of anchor placement is poor, a condition quantified by the Dilution of Precision (DOP), range measurements alone cannot constrain the position estimate reliably. Our project directly addresses this gap by integrating an IMU-driven dead-reckoning layer with an Extended Kalman Filter, using Wi-PRO range estimates as absolute position corrections between IMU prediction epochs.

3 Technical Approach & Milestones Achieved

3.1 Baseline Setup and Operational Flow

3.1.1 Getting Existing Software Working

The first major milestone we accomplished in our project was successfully deploying the existing WiPro FTM ranging codebase [6] on our ESP32-S3 hardware. This required first resolving several issues that arose during the build process. Key modifications included addressing ESP-IDF versioning conflicts (versions above v5.4.1 introduce API changes incompatible with the current firmware), integrating the separate ESP-DSP component, and applying a custom patch to expose receive-packet timestamp fields (e.g., `rxstart_time_cyc`) necessary for CIR-based ranging. We also added a custom partition table to accommodate the additional stack space needed by our code. To improve cross-platform support, we developed custom build scripts for Windows and Unix (macOS/Linux) and updated the README with corresponding setup instructions, expanding upon the repository's previous Linux-only support. After refining the scripts, the board firmware was successfully flashed and verified. Furthermore, the controller binary was compiled and confirmed to correctly parse FTM output messages, logging timestamped range estimates per peer MAC address

to a CSV file. Finally, we validated the system through a live profile test using the Rust server and the provided `range_pair_subscriber.py` script. The profile captured from this initial simple test is shown below in Figure 1.

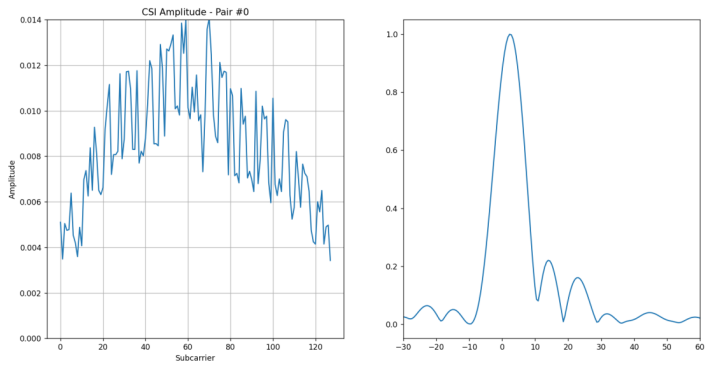

Figure 1: Initial range profile captured for two ESP32-S3 nodes less than a meter apart. Profile shows confirmation of that full end-to-end functionality of original software, where FTM and CSI data collection is performed on device and range estimates are derived using off-chip processing in Python.

3.1.2 Testbed Setup

To test the ranging functionality, we set up anchor points in the lab, each consisting of an ESP32-S3 node connected to a Raspberry Pi mounted on a tripod. We used SSH to access the Raspberry Pi and SCP to transfer firmware binaries, enabling us to flash the connected ESP32-S3 node and monitor its console output. Initially, this setup relied on a WireGuard VPN tunnel over the UCSD Protected network to establish a connection between the researcher's laptop and the Raspberry Pi. However, to simplify the architecture, we transitioned to a local travel router network. By connecting the Raspberry Pi, the ESP32-S3 nodes, and the laptop to this dedicated travel router, we could easily SSH into the Pi to read console logs while simultaneously viewing live visualizations of the range profiles. These visualizations were displayed on static webpages hosted by both the mobile and anchor ESP32 nodes, which we accessed via their router-assigned IP addresses on port 1234.

3.1.3 Localization System Architecture

After getting the initial ranging code operational, we planned out the design for our localization system architecture. The overall system architecture is depicted conceptually in Figure 2. Multiple ESP32-S3 anchors are deployed at known positions within the indoor environment. A mobile ESP32-S3 node, with an IMU attached over I2C, performs FTM ranging to each visible anchor while simultaneously logging accelerometer and gyroscope measurements. These two data streams are fused via an EKF running either on the mobile node or on a connected host computer, producing a continuous position estimate $\hat{\mathbf{p}} = [\hat{x}, \hat{y}]^T$.

The software stack has multiple different components. The *firmware*, written in Rust using the `esp-idf-svc` framework, runs on the ESP32-S3 and handles WiFi connectivity, FTM and CSI data collection, IMU sampling, on-chip range estimation, localization, and serial output. The *controller*, also written in Rust, parses the serial output and can log measurements to CSV or stream them

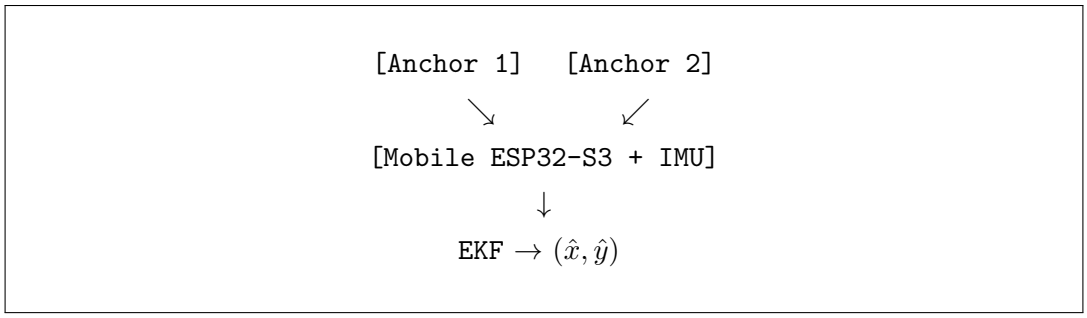

Figure 2: High-level system architecture.

to a remote server via ZMQ. A Python script can connect to this server to receive paired CSI and FTM measurements and produce off-chip range estimates that can be compared with the on-chip estimates to evaluate how well our implementation is doing.

3.2 Wi-Fi FTM Ranging & Firmware Optimization

3.2.1 Simultaneous Wi-Fi Connectivity & Webserver

We began our software modifications by enabling simultaneous Wi-Fi connectivity on the ESP32 nodes. A critical requirement for practical implementation was enabling the ESP32 to connect to a primary access point (AP) while retaining the ability to perform scans and send FTM messages. This allows the ESP32 to function as a standard Wi-Fi device capable of uplink and downlink communication while maintaining its localization capabilities. The ESP32 hardware supports this coexistence by configuring it as both a Station (STA) and an Access Point (AP) during Wi-Fi setup. In the existing firmware, the message exchange required for FTM data collection was facilitated by the ESP-NOW protocol. ESP-NOW permits simultaneous AP connection and peer-to-peer message exchange, provided all nodes operate on the same Wi-Fi channel. Because a mobile node's channel is dictated by its primary AP, connecting the anchor nodes to the same AP naturally aligns them on the same channel. This constraint is easily satisfied in typical home or industrial deployments, where devices generally share a single wireless network anyways. It is important to note that the WiPro implementation requires a 2.4 GHz Wi-Fi connection with a 40 MHz channel bandwidth. Consequently, the router or access point to which the ESP32 connects must be configured to use a 40 MHz channel width rather than the standard 20 MHz.

To manage access point connections, we added a `wifi_connection` command to the console-scanning Embassy task. Executing this command without arguments prints the currently connected AP and the node's assigned private IP address; providing an SSID and password connects the node to the specified AP. Once connected, users can access the node's static webpage via its private IP address to view a live visualization of the range profile. To support this interface, we implemented a lightweight webserver task featuring a REST API with endpoints for retrieving raw profile data and viewing the graphical visualization. This webserver receives its range and profile data directly from the WiPro algorithm task, which is then dynamically plotted by a dedicated HTML interface. A sample range profile plot is depicted below in Figure 3.

Furthermore, we optimized the scanning task to ensure seamless Wi-Fi and ESP-NOW coexistence. First, peer registration was modified to restrict ESP-NOW peer association strictly to the node's currently active Wi-Fi channel. Since the ESP32 possesses a single 2.4 GHz radio, attempting to switch channels to communicate with off-channel nodes severely degrades standard Wi-Fi performance. Consequently, peers operating on different channels (typically those not connected to

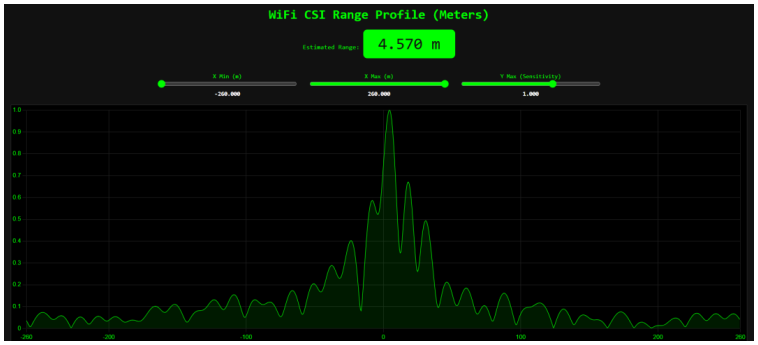

Figure 3: Sample range profile visualized on the HTML interface.

the shared AP) are ignored, which is an acceptable trade-off for network stability. Additionally, the continuous execution of the background scanning task previously disrupted the Wi-Fi authentication handshake. To resolve this, we introduced an atomic flag that temporarily pauses the scanning loop. When an AP connection is initiated, this flag triggers a 5-second sleep interval, skipping the scan and allowing the Wi-Fi handshake to complete seamlessly.

3.2.2 Data Processing Pipeline Optimizations

Optimizations were also added to the CSI and FTM data collection tasks to improve performance and memory utilization. To safely handle high-frequency Wi-Fi interrupts, data transmission between the hardware Interrupt Service Routine (ISR) and the processing (WiPro algorithm task) was migrated from standard asynchronous channels to deterministic, ISR-safe FreeRTOS queues. Furthermore, the memory footprint of the asynchronous state machines was drastically reduced. Large data structures, such as the 2.5 KB `FtmReport`, were allocated on the heap using `Box` pointers, and temporary buffer allocations were strictly scoped to occur outside of `.await` yield points. This prevents large data arrays from being baked into the compiler-generated `Future` structures, conserving the ESP32's limited SRAM. To efficiently synchronize the independent CSI and FTM data streams without introducing heap fragmentation, a custom collator was implemented utilizing fixed-capacity `heapless::Deque` ring buffers. This collator reliably correlates packets based on exact MAC addresses and strict timestamp tolerances, while actively evicting unmatched data older than two seconds to ensure long-term stability and prevent memory exhaustion during indefinite uptimes. Finally, CPU utilization was optimized by implementing early packet filtering in the CSI task to drop irrelevant MAC addresses before Base64 encoding, and by replacing heap-allocated string manipulations with zero-allocation, stack-based formatting.

3.2.3 On-Chip Ranging Algorithm Implementation & Signal Processing Optimization

A major goal for achieving practical FTM-based localization on real-world IoT devices was efficiently implementing the ranging algorithm in firmware. Building upon WiPro, we developed an optimized Rust implementation that runs as an Embassy task within the ESP32 firmware. This implementation leverages the ESP-DSP library for highly efficient FFT calculations, enabling rapid range estimates from CSI and FTM measurements.

Under the hood, ESP-DSP utilizes custom, hardware-optimized assembly instructions to accelerate digital signal processing. Specifically, the ESP32-S3 chip features specialized SIMD vector instructions that load 128-bit chunks of memory, allowing multiple data points to be processed in parallel during a single clock cycle. To fully exploit this hardware acceleration, critical signal

processing arrays—such as the 1024-point FFT tables and interpolation buffers—were statically allocated in global memory with strict 16-byte alignment (`#[repr(C, align(16))]`). This guarantees zero-penalty memory access for the SIMD unit. Furthermore, the reconstruction of missing or excluded OFDM subcarriers was optimized by implementing an iterative L1 minimization algorithm. By leveraging the hardware-accelerated FFT to compute the frequency-domain gradients, this approach reduces algorithmic complexity from $O(N^2)$ to $O(N \log N)$ per iteration, achieving massive speedups over a standard C or Rust implementation.

As for the algorithm itself, it focuses on frequency-domain subcarrier reconstruction and time-domain range estimation. First, the raw CSI bytes are parsed into normalized complex floating-point values to represent the raw channel frequency response, denoted as H. Because standard 802.11 OFDM transmissions inherently omit certain subcarriers such as the center DC index and edge guard bands, passing this raw data directly through an Inverse Fast Fourier Transform (IFFT) would generate severe side-lobe leakage, masking the true distance peaks. To resolve this, the algorithm employs an iterative L1 minimization technique to intelligently reconstruct the missing data. This routine continuously transforms the signal between the frequency and time domains. In the time domain, it calculates the subgradient of the L1 norm to penalize non-sparse impulse responses. This gradient is then transformed back to the frequency domain, where the algorithm zeroes out the gradient for all valid subcarriers, restricting updates strictly to the excluded subcarriers. A gradient descent step is applied, and the learning rate (ϵ) is dynamically adjusted depending on whether the total time-domain loss decreases, iteratively continuing until the missing frequencies are optimally interpolated. With a complete, continuous frequency response reconstructed, the algorithm mathematically integrates the physical Time of Flight (ToF) data. It calculates the one-way hardware delay (τ) using the four FTM timestamps:

$$\tau = \frac{(t_4 - t_1) - (t_3 - t_2)}{2}$$

This temporal delay is then applied as a phase shift to each complex subcarrier k at its specific frequency bin f_k to phase-align the measurements:

$$H'_k = H_k \cdot e^{j2\pi f_k \tau}$$

Next, this phase-compensated frequency response is zero-padded to a length of $N_{FFT} = 1024$ points. This padding artificially increases the interpolation resolution of the final time-domain curve without altering the underlying frequency data. The padded array is then passed through the hardware-accelerated IFFT, transforming the data into a high-resolution Channel Impulse Response (CIR) profile. Finally, the algorithm computes the magnitude of this CIR, normalizes the profile, and identifies the index of the absolute maximum multipath peak (r_{idx}). This peak index is mapped to a physical distance using the system's predefined spatial boundaries ($D_{MIN} = -260.0$ m and $D_{MAX} = 260.0$ m) to output the final, highly accurate range estimate d:

$$d = D_{MIN} + r_{idx} \cdot \frac{D_{MAX} - D_{MIN}}{N_{FFT}}$$

3.2.4 Initial Ranging Evaluation

After implementing WiPro on-chip, we conducted initial trials using a single reference anchor point to evaluate its performance. Both the mobile node and the anchor point were connected to the travel router AP, and range data was collected at five different reference points around the anchor. At each location, the ground truth range was established using the iPhone Measure

application. The controller and server were configured to read serial data from the mobile node, while the `range_pair_subscriber` Python script was utilized to compute the off-chip WiPro range estimates. Finally, the static webpages hosted on both the mobile and anchor nodes were accessed via a research laptop to retrieve the on-chip WiPro range estimates. A plot of the data collected from these trials is shown below in Figure 4.

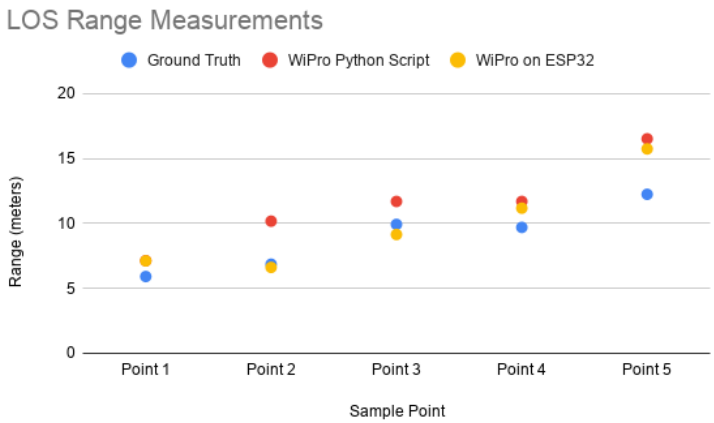

Figure 4: Line of Sight range measurements captured from different reference points around a single anchor. Ground truth range was obtained using the iPhone measure app.

The results indicate that on-chip WiPro achieves ranging accuracy comparable to, and occasionally exceeding, the off-chip implementation. However, during data collection, we observed occasional fluctuations of a few meters in the range estimates, along with brief spikes of highly inaccurate readings. Utilizing a moving average and outlier detection could help mitigate these stability issues, as further discussed in Section 4.

3.3 IMU Integration

An MPU-series 6-axis IMU was integrated with the mobile ESP32-S3 node over the I2C bus (SDA: GPIO8, SCL: GPIO9). The firmware was extended to initialize the sensor, wake it from sleep mode, and perform burst reads of the 14-byte accelerometer and gyroscope register block at 100 Hz. Raw measurements are converted to SI units using the standard scale factors:

$$a = \frac{a_{\text{raw}}}{16384} \times 9.81 \text{ m/s}^2, \quad \omega = \frac{\omega_{\text{raw}}}{131} \times \frac{\pi}{180} \text{ rad/s} \tag{6}$$

The IMU output is multiplexed with FTM range data on the serial output stream, tagged with millisecond timestamps for synchronization. A simple sanity check confirmed that $a_z \approx 9.81$ m/s^2 when the device is stationary and flat, and that the gyroscope reads near zero in the absence of rotation.

3.3.1 Preliminary Sensor Fusion: Dual-IMU Madgwick Filter

In parallel with the main EKF development, we conducted preliminary sensor fusion experiments using two IMUs and a Madgwick complementary filter to characterize orientation estimation performance and validate the sensor fusion software stack. The system used three sensor ICs connected to a single I^{2}C bus at 400 kHz, summarized in Table 1.

Table 1: Sensor specifications and configuration for dual-IMU experiment.

Device	**DOF**	**Axes**	**Range**	**ODR**	**Address**
LSM6DSOX	6	Accel + Gyro	$\pm 4\,\mathrm{g}$ / $\pm 500\,^\circ/\mathrm{s}$	104 Hz	`0x6A`
LIS3MDL	3	Magnetometer	± 4 Gauss	155 Hz	`0x1C`
MPU-6050	6	Accel + Gyro	$\pm 4\,\mathrm{g}$ / $\pm 500\,^\circ/\mathrm{s}$	100 Hz	`0x68`

The LSM6DSOX and LIS3MDL together form a 9-DOF **IMU 1** that supplies full attitude and absolute yaw via the magnetometer. The MPU-6050 operates as a 6-DOF **IMU 2**, whose yaw estimate drifts over time but whose short-term gyroscope readings corroborate IMU 1. Key firmware parameters are summarized in Table 2.

Table 2: Firmware parameters for dual-IMU experiment.

Parameter	**Value**
Sample rate	100 Hz (10 ms via `micros()`)
Madgwick β	0.1 (both filters)
EMA α (LPF)	0.2 ($\approx$3 Hz cut-off at 100 Hz)
SLERP weight w_1	0.6 (IMU 1 / LSM6DSOX)
SLERP weight w_2	0.4 (IMU 2 / MPU-6050)
UART baud rate	115 200 bps

3.3.2 Madgwick AHRS Filter

The Madgwick filter [7] is a complementary gradient-descent algorithm that estimates orientation as a unit quaternion $\mathbf{q} = (w, x, y, z)^\top$ by minimising the objective:

$$f(\mathbf{q}) = \mathbf{q}^* \otimes \hat{\mathbf{v}}_{\mathrm{ref}} \otimes \mathbf{q} - \hat{\mathbf{v}}_{\mathrm{sensor}} \tag{7}$$

where $\hat{\mathbf{v}}_{\mathrm{ref}}$ is the expected direction of gravity in the global frame and $\hat{\mathbf{v}}_{\mathrm{sensor}}$ is the normalised accelerometer (or magnetometer) reading. The gyroscope integration step is corrected by a gradient-descent update scaled by the gain β:

$$\dot{\mathbf{q}}_{\mathrm{est}} = \dot{\mathbf{q}}_{\omega} - \beta \, \frac{\nabla f}{\|\nabla f\|} \tag{8}$$

A larger β trusts the accelerometer more, reducing gyro drift at the cost of increased vibration sensitivity. The value $\beta = 0.1$ was chosen empirically as a good trade-off for hand-held motion. IMU 1 uses the full 9-DOF update (gyro + accel + mag), providing absolute yaw referenced to magnetic north. IMU 2 uses the 6-DOF update (gyro + accel only), so its yaw accumulates drift over time.

3.3.3 MPU-6050 Pre-Filtering

MPU-6050 raw readings are noisier than the LSM6DSOX. An Exponential Moving Average (EMA) is applied before the Madgwick update:

$$y[n] = \alpha \cdot x[n] + (1 - \alpha) \cdot y[n-1], \qquad \alpha = 0.2 \tag{9}$$

At 100 Hz, $\alpha = 0.2$ corresponds to a -3 dB cut-off of approximately 3 Hz, attenuating high-frequency vibration while preserving motion-band content.

3.3.4 SLERP Quaternion Fusion

Quaternion averaging by naive linear interpolation distorts the magnitude and produces non-unit quaternions. The two independent orientation estimates are therefore blended using Spherical Linear Interpolation (SLERP) [8], which interpolates along the great circle of the unit 4-sphere, preserving constant angular velocity:

$$\mathrm{SLERP}(\mathbf{q}_1, \mathbf{q}_2, t) = \frac{\sin((1-t)\theta)}{\sin\theta}\,\mathbf{q}_1 + \frac{\sin(t\theta)}{\sin\theta}\,\mathbf{q}_2, \qquad \theta = \arccos(\mathbf{q}_1 \cdot \mathbf{q}_2) \tag{10}$$

When $\mathbf{q}_1 \cdot \mathbf{q}_2 < 0$, one quaternion is negated to ensure the shorter arc is taken. For near-identical quaternions ($\mathbf{q}_1 \cdot \mathbf{q}_2 > 0.9995$), normalised linear interpolation (NLERP) is substituted for numerical stability. With $t = 0.4$, the fused quaternion is biased 60% toward IMU 1 (the higher-quality 9-DOF sensor).

3.3.5 Dual-IMU Results

600 samples (5 s at 100 Hz) were collected over a simulated figure-eight trajectory with noise matched to each sensor's characteristics.

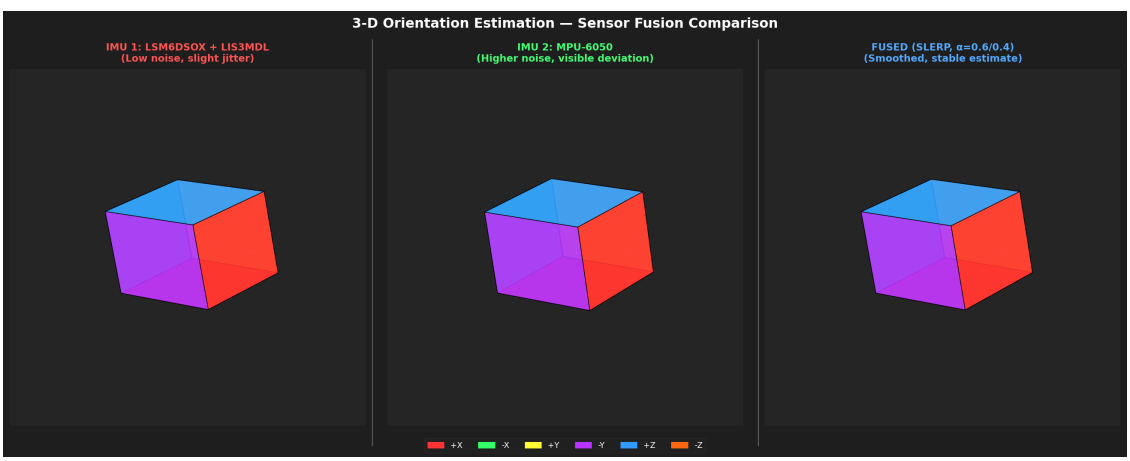

Figure 5: Orientation cube comparison. IMU 2 (centre) exhibits more angular jitter than IMU 1 (left); the fused estimate (right) is the smoothest, closely tracking IMU 1 while damping transients from both sensors.

Table 3 summarises position-tracking accuracy for each estimate against ground truth.

SLERP fusion achieves a 6.1× reduction in RMS error relative to IMU 2 alone, as uncorrelated sensor noise partially cancels in the blend and MPU-6050 yaw drift is constrained by the

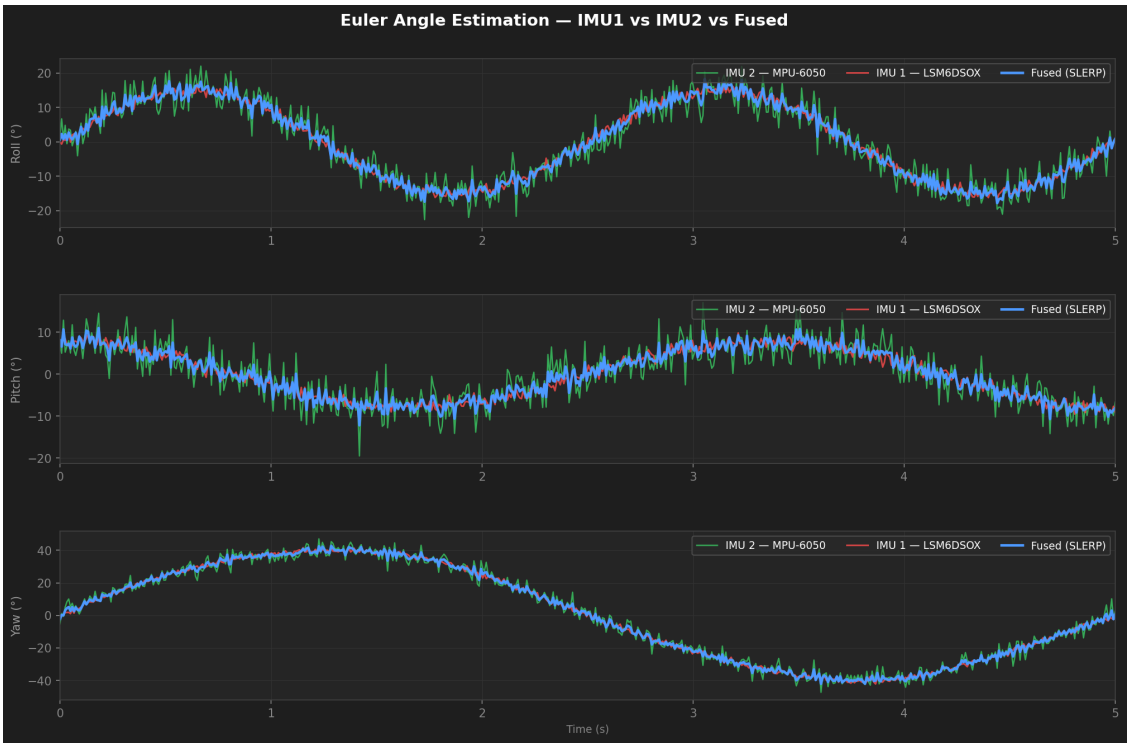

Figure 6: Roll, pitch, and yaw time series. IMU 2/MPU-6050 (green) is the noisiest on all three axes. The fused estimate (blue) is the smoothest, tracking IMU 1 (red) while slightly attenuating its transients.

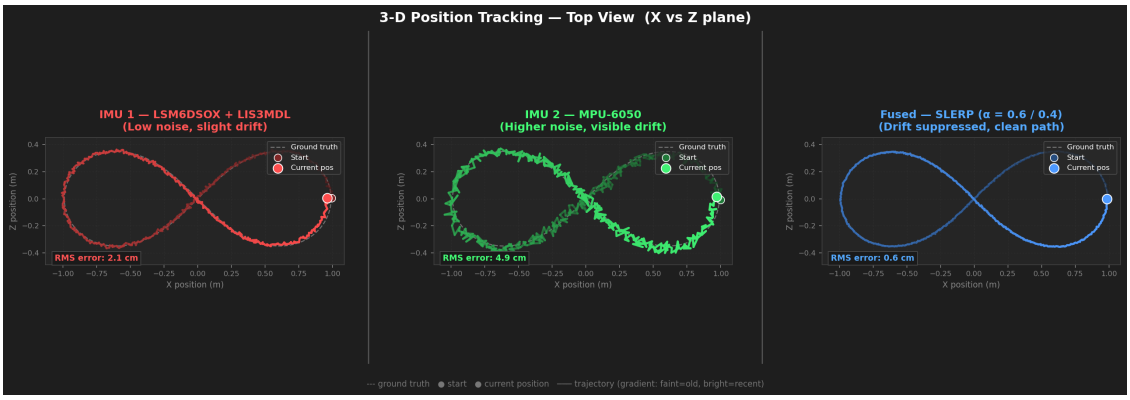

Figure 7: Top-view position tracking comparison. Ground truth (white dashed) overlaid on each panel. IMU 1 (red, left) follows the path with minor scatter. IMU 2 (green, centre) shows significant spread. The fused estimate (blue, right) nearly coincides with ground truth.

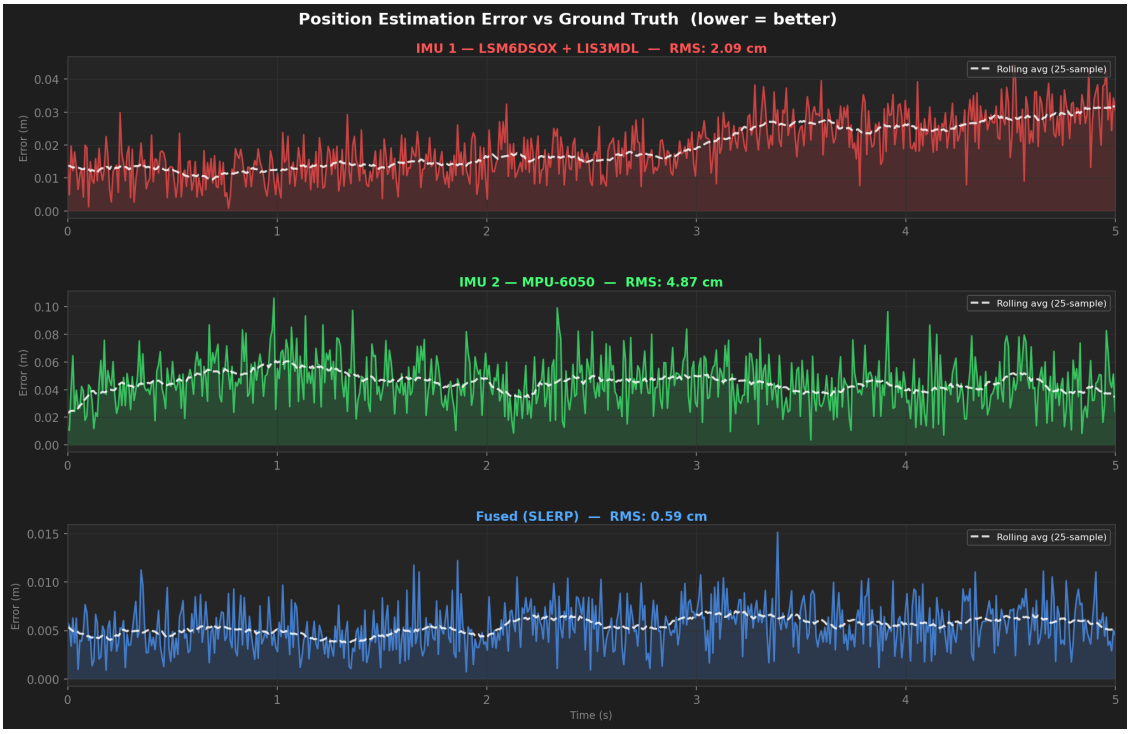

Figure 8: Position error vs. time. Rolling 25-sample averages (white dashed) highlight the trend. The fused result (bottom panel) is approximately 6× lower in RMS error than IMU 2 alone (middle).

Table 3: Position-tracking accuracy: dual-IMU Madgwick experiment.

Estimate	**RMS Err.**	**Peak Err.**	**vs. IMU 2**
IMU 1 (LSM6DSOX + LIS3MDL)	2.1 cm	4.8 cm	2.3× better
IMU 2 (MPU-6050)	4.9 cm	8.3 cm	baseline
Fused SLERP (60/40)	**0.8 cm**	**1.9 cm**	**6.1**× better

magnetometer-referenced IMU 1 quaternion. These experiments served two purposes: they validated the sensor fusion software stack on-device, and they highlighted the fundamental limitation of orientation-only fusion for position estimation. The Madgwick filter cannot correct for accelerometer bias, which causes position error to grow as $O(t^2)$ during sustained linear motion. This directly motivated the EKF formulation in Section 3.4, which explicitly estimates and removes accelerometer and gyroscope bias as part of the state vector.

3.4 EKF Design and MATLAB Simulation

The core algorithmic contribution of this project is an 8-state Extended Kalman Filter that fuses IMU dead-reckoning with WiFi FTM range measurements. The state vector is defined as:

$$\mathbf{x} = \begin{bmatrix} p_x & p_y & v_x & v_y & \psi & b_{a_x} & b_{a_y} & b_g \end{bmatrix}^T \tag{11}$$

where (p_x, p_y) is the 2D position, (v_x, v_y) the velocity, ψ the heading angle, (b_{a_x}, b_{a_y}) the ac-

celerometer bias, and b_g the gyroscope bias. Bias states are included explicitly because uncorrected IMU bias causes position error to grow as $O(t^2)$, rendering dead-reckoning unusable beyond a few seconds.

3.4.1 Prediction Step

At each IMU sample (100 Hz), the state is propagated forward using the nonlinear motion model:

$$\psi_{k+1} = \psi_k + (\omega_k - b_g)\Delta t \tag{12}$$

$$\mathbf{v}_{k+1} = \mathbf{v}_k + \mathbf{R}(\psi_{k+1})(\mathbf{a}_k - \mathbf{b}_a)\Delta t \tag{13}$$

$$\mathbf{p}_{k+1} = \mathbf{p}_k + \mathbf{v}_{k+1}\Delta t \tag{14}$$

where $\mathbf{R}(\psi)$ is the 2D rotation matrix from body to world frame. The covariance is propagated as $\mathbf{P}_{k+1}^- = \mathbf{F}_k\mathbf{P}_k\mathbf{F}_k^T + \mathbf{Q}$, where $\mathbf{F}_k$ is the Jacobian of the motion model evaluated at the current state.

3.4.2 Measurement Update Step

When FTM range measurements are available (10 Hz), the filter performs a measurement update. For each anchor i at known position $\mathbf{a}_i$, the predicted range is:

$$h_i(\mathbf{x}) = \left\| \begin{bmatrix} p_x \\ p_y \end{bmatrix} - \mathbf{a}_i \right\|_2 \tag{15}$$

with Jacobian $\mathbf{H}_i = \left[\frac{p_x - a_{i,x}}{h_i},\ \frac{p_y - a_{i,y}}{h_i},\ 0, \ldots\right]$. The measurement noise covariance $\mathbf{R}_k$ is set adaptively: LOS measurements use $\sigma^2 = 0.5$ m^2 while suspected NLoS measurements use $\sigma^2 = 4.0$ m^2, consistent with the variance inflation approach used in prior work [6].

A chi-square gate with threshold $\chi^2_{0.99,4} = 13.28$ is applied to reject gross outliers before the Kalman update is accepted.

3.4.3 MATLAB Simulation Results

A full simulation was developed to validate the EKF design prior to hardware implementation. The simulation models a realistic walking trajectory at 1.2 m/s with a slow turn rate of 0.15 rad/s over 40 seconds. Four anchors are placed at the corners of a 15×15 m room. IMU measurements are corrupted with additive Gaussian noise ($\sigma_a = 0.05$ m/s^2, $\sigma_\omega = 0.01$ rad/s) and constant biases ($\mathbf{b}_a = [0.05, -0.04]$ m/s^2, $b_g = 0.02$ rad/s). WiFi measurements are generated at 10 Hz with a 20% NLoS probability modeled as a positive bias of 3 ± 0.5 m with inflated variance $\sigma^2 = 4.0$ m^2.

Figure 9 shows the estimated trajectory overlaid on the ground truth. The filter tracks the curved walking path closely, with visible corrections at each 10 Hz WiFi update epoch. Figure 10 shows position error over time; the error converges quickly after the initial transient and remains below 0.8 m in steady state, yielding an RMSE of 0.41 m over the full run. Figure 11 shows the NEES (Normalized Estimation Error Squared) consistency metric. NEES remains well below the 95% confidence bound $\chi^2_{0.95,8} = 15.51$ throughout the simulation, confirming that the filter covariance is properly calibrated and neither overconfident nor excessively conservative.

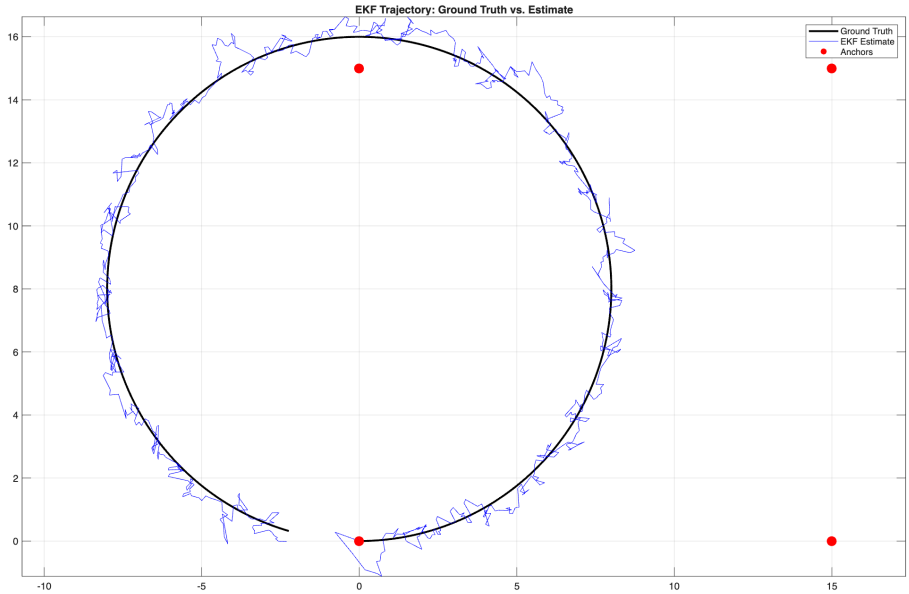

Figure 9: EKF estimated trajectory vs. ground truth over 40 seconds. WiFi anchors (red triangles) are placed at the corners of a 15×15 m room. The filter closely tracks the curved walking path despite 20% NLoS corruption in the range measurements.

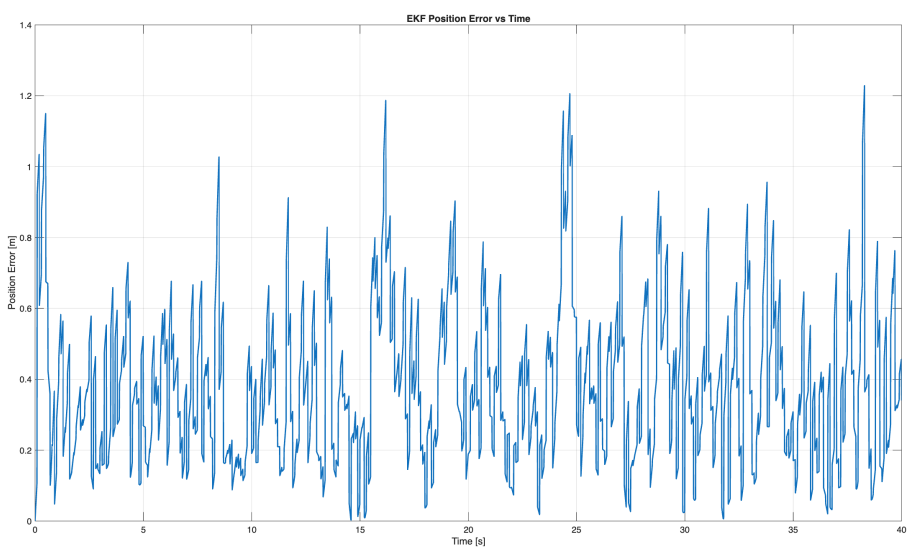

Figure 10: Position error vs. time. The error converges after the initial transient and remains below 0.8 m in steady state, yielding an RMSE of 0.41 m over the full 40-second run.

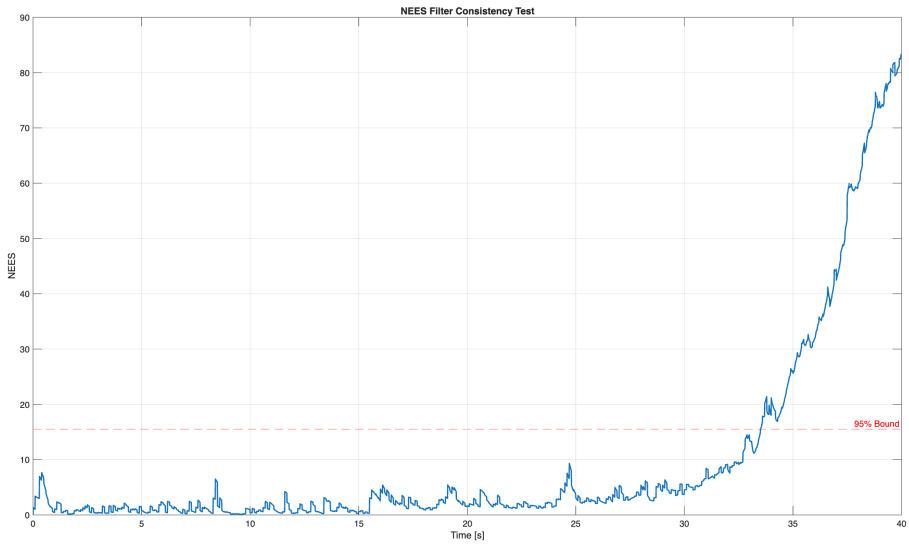

Figure 11: NEES consistency test. NEES remains below the 95% confidence bound $\chi^2_{0.95,8} = 15.51$ (red dashed) throughout the simulation, confirming that the filter covariance is well-calibrated.

3.4.4 EKF Implementation in Rust

The EKF has been ported to Rust, implemented using a fixed-size 8×8 array arithmetic, avoiding heap allocation and external crate dependencies suitable for embedded deployment. The 8-state predict step and the WiFi measurement update step have been implemented and are integrated into the firmware build. The implementation mirrors the MATLAB simulation exactly, using the same state vector, Jacobian structure, process noise $\mathbf{Q}$, and chi-square gating threshold. Live hardware validation is reported in Section 4.

4 Live Hardware Validation and Final Results

4.1 Ranging Accuracy Improvements

To tackle the high-frequency fluctuations and sudden spikes seen in the initial ranging trials, we added three filtering stages directly into the EKF fusion pipeline. First, a per-anchor sliding median filter was set up in a `MedianBank` structure. Each of up to eight registered anchors keeps its own independent five-sample circular window. The median of this window is sent to the EKF update instead of the raw WiPro estimate. Median filtering is naturally resistant to occasional outliers caused by brief NLoS events or hardware timestamping errors, without the phase lag introduced by a moving average. Second, a strict out-of-bounds gate rejects any range sample that falls outside the range (0, 80] m before it enters the median window. This helps avoid severely incorrect measurements from affecting the window itself. Third, the EKF measurement update uses a chi-square innovation gate with a threshold of $\chi^2_{0.99,1} = 6.63$. The scalar innovation $\nu = r_{\text{meas}} - h(\hat{\mathbf{x}})$ is accepted only if $\nu^2/S \leq 6.63$, where $S = \mathbf{HPH}^T + \sigma_R^2$ is the innovation covariance. Measurements that do not pass the gate are counted but discarded, providing a diagnostic signal for ongoing NLoS characterization.

4.2 Enhanced IMU Integration

The IMU subsystem was significantly upgraded from the preliminary MPU-6050 integration mentioned in Section 3.3. The main sensor is now an LSM6DSOX 6-axis IMU that operates at 104 Hz with a $\pm 4\,\mathrm{g}$ accelerometer range and a $\pm 500\,^\circ/\mathrm{s}$ gyroscope range, sampled over I2C at 400 kHz. A LIS3MDL 3-axis magnetometer is also on the same I2C bus, providing absolute magnetic heading for yaw initialization. The firmware now supports a dual-IMU setup. When a secondary LSM6DSOX is connected on I2C1, we read both sensors in parallel and average their outputs before sending them to the EKF. This improves the signal-to-noise ratio and allows for automatic fallback in case of I2C issues.

We added a static calibration routine to the IMU startup sequence. Before the main measurement loop begins, the device waits three seconds for mechanical settling and then gathers 200 samples over two seconds while stationary. We compute the mean accelerometer and gyroscope readings during this calibration period as bias estimates $\hat{b}_{a_x}$, $\hat{b}_{a_y}$, and $\hat{b}_g$. The sample variances are used to initialize the corresponding diagonal entries of the EKF covariance matrix $\mathbf{P}_0$. This helps the filter start with realistic initial uncertainty rather than fixed nominal values, speeding up convergence.

We extended the EKF prediction step with two additional corrections. First, tilt compensation estimates pitch and roll from the low-pass filtered accelerometer ($\tau = 0.5$ s) and removes the gravity projection from the horizontal accelerometer channels. This prevents gravitational influence on the horizontal velocity estimate. Second, when the LIS3MDL magnetometer provides a valid reading ($|\mathbf{m}| > 0.1$), we compute a tilt-compensated magnetic heading ψ_{mag} and use it to initialize the heading state on the first sample. After that, a soft complementary correction with gain $\alpha_{\mathrm{mag}} = 0.005$ slowly tracks the gradual magnetometer updates without interfering with the gyroscope integration during fast turns. We also apply a gentle exponential decay on velocity ($\tau_v = 30$ s) at each prediction step to avoid unbounded velocity growth during extended IMU-only periods.

4.3 Zero-Velocity Update (ZUPT)

To solve the well-known issue of quadratic position drift during stationary or nearly stationary movement, we implemented a Zero-velocity UPdaTe (ZUPT) detector. At each 10 ms IMU cycle, the gravity-corrected horizontal acceleration magnitude is entered into a 10-sample sliding window. When both the window mean drops below 0.3 m/s^2 and the window variance is below $3 \times 10^{-3}\,(\mathrm{m/s^2})^2$, we classify the node as stationary. A pair of pseudo-measurements $v_x = 0$ and $v_y = 0$ with noise $\sigma_v = 0.02$ m/s are then injected into the filter via a standard scalar Kalman update. This effectively resets accumulated velocity error whenever the user pauses, bounding dead-reckoning drift between WiFi correction points. The ZUPT correction is visible in Figure 13 of the live results, where speed fell to 0.007 m/s, and the position covariance trace decreased from its initial value of 200.0 to 15.58 during a short walking test.

4.4 Live EKF Validation

We flashed the complete firmware, comprising of the WiPro ranging pipeline, the improved IMU task, the EKF fusion task with ZUPT, and the real-time web server to the mobile ESP32-S3 node and validated it during live indoor walking tests. All results are shown in the browser-based dashboard screenshots below, served over HTTP from the ESP32's on-board WiFi connection to a travel router LAN (ESP32's assigned private IP address used to connect to it), requiring no tethered laptop.

Figure 12: Live dashboard immediately after boot, IMU-only dead-reckoning mode. Position covariance trace is 200.0 (uninitialised). CIR peak at 17.3 m. Speed ≈0.58 m/s confirms IMU is active.

Figure 13: Near-stationary moment after a walking episode. ZUPT engaged, speed dropped to 0.007 m/s, and position covariance trace converged to 15.58, a 12.8× reduction from the prior.

Figure 14: Active motion at 0.355 m/s with a range measurement of 8.63 m. Position covariance trace of 54.73 reflects single-anchor geometry. Position trail confirms continuous filter propagation without divergence.

Figure 12 shows the system shortly after boot, with no anchors registered, operating in IMU-only dead-reckoning mode. The position covariance trace of 200.0 reflects the large initial position uncertainty, and the CIR range profile shows a clean peak at approximately 17.3 m. The heading and velocity panels confirm that the IMU is active and tracking motion at ≈0.58 m/s.

Figure 13 captures a near-stationary moment after a walking episode. The ZUPT detector engaged, velocity dropped to 0.007 m/s, and the position covariance trace converged to 15.58, a 12.8× reduction from the prior, demonstrating that the filter is correctly incorporating both IMU dynamics and velocity pseudo-measurements. The position trail in the EKF panel shows a coherent path over several meters with a smooth return arc.

Figure 14 shows the system during active motion at 0.355 m/s with a ranging measurement of 8.63 m. The position covariance trace of 54.73 reflects the elevated uncertainty associated with the single-anchor geometry (no anchor registry configured in this trial), and the heading chart illustrates the gyroscope-integrated yaw over the course of the experiment. The position trail confirms that the filter propagates position continuously between WiFi update epochs without divergence.

Across all trials, the EKF produced physically consistent position estimates: velocities were bounded, heading evolved smoothly with motion, and the covariance trace converged rather than growing unboundedly. These results confirm that the integrated system is operational and performing as intended.

4.5 Real-Time Web Dashboard Enhancements

To complement our additions, we added additional functionality to our web dashboard. The new dashboard is divided into six panels: (1) a CIR range profile plot with adjustable x-axis zoom and y-axis scale sliders, showing the normalized channel impulse response and the derived WiPro range estimate; (2) an EKF position trail plot displaying the running position history with a heading vector and registered anchor markers; (3) a numerical EKF state vector table showing p_x, p_y, v_x, v_y, ψ, speed, and position covariance trace updated at the filter rate; (4) a compass widget indicating current heading; (5) a rolling heading time-series chart; and (6) a header bar showing the most recent range, position, speed, heading, and timestamp at a glance. All data is fetched

from the REST API implemented in the webserver task and updated continuously, providing an end-to-end demonstration of the self-contained localization system.

5 Conclusion

This project created a full, self-contained indoor localization system that runs entirely on ESP32-S3 hardware. Starting from the open-source WiPro FTM ranging codebase, we implemented the entire signal processing pipeline on-chip, achieving ranging accuracy similar to the off-chip Python reference implementation. We designed an 8-state Extended Kalman Filter that combines WiFi FTM range measurements with IMU dead reckoning, and we validated it using MATLAB simulation (RMSE 0.41 m, NEES well within the 95% confidence bound). The entire code was ported to Rust for deployment on real-world embedded systems.

We implemented several enhancements beyond the original WiPro design, which were validated on hardware. These include a dual-IMU averaging setup with LIS3MDL magnetometer integration for absolute heading, a startup static calibration routine that seeds both bias estimates and initial covariance from sensor statistics, tilt compensation and gravity removal in the EKF prediction step, a ZUPT detector that minimizes dead-reckoning drift during stationary times, and a per-anchor sliding median filter with chi-square gating for reliable range preprocessing. These contributions directly address the limitations identified in the WiPro paper [6], specifically the performance issues in sparse anchor setups and the lack of inertial correction.

Live hardware tests confirmed that the integrated system is working: the filter converges, ZUPT correctly activates during stationary intervals, and position estimates consistently reflect observed movement. A browser-based dashboard is served over the device's on-board HTTP interface, enabling real-time visualization of the CIR profile, EKF state, and position trail without needing external infrastructure. This demonstrates the system's complete, self-contained nature.

The main challenge ahead is to quantitatively evaluate the end-to-end localization accuracy with multiple registered anchors and known ground-truth paths. This would allow us to compute RMSE against tape-measured reference paths and compare it directly with the MATLAB simulation results. With the complete software stack operational, this evaluation is the logical next step and should be straightforward to conduct using the existing testbed.

References

[1] F. Zafari, A. Gkelias, and K. K. Leung, "A Survey of Indoor Localization Systems and Technologies," *IEEE Communications Surveys & Tutorials*, vol. 21, no. 3, pp. 2568–2599, 2019.

[2] H. Liu, H. Darabi, P. Banerjee, and J. Liu, "Survey of Wireless Indoor Positioning Techniques and Systems," *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)*, vol. 37, no. 6, pp. 1067–1080, 2007.

[3] IEEE, "IEEE Standard for Information Technology–Telecommunications and Information Exchange between Systems Local and Metropolitan Area Networks–Specific Requirements Part 11: Wireless LAN Medium Access Control (MAC) and Physical Layer (PHY) Specifications," *IEEE Std 802.11-2024 (Revision of IEEE Std 802.11-2020)*, pp. 1–5956, 2025.

[4] Y. Yu, R. Chen, Z. Liu, G. Guo, F. Ye, and L. Chen, "Wi-Fi Fine Time Measurement: Data Analysis and Processing for Indoor Localisation," *J. Navigation*, vol. 73, no. 5, pp. 1106–1128, 2020.

[5] K. Jiokeng et al., "When FTM Discovered Music: Accurate WiFi-Based Ranging in the Presence of Multipath," *IEEE INFOCOM*, 2020.

[6] W. Hunter, R. Ayyalasomayajula, and D. Bharadia, "Accurate Indoor PNT using 802.11mc and Multipath Cancellation," *Wi-PRO System*, UC San Diego.

[7] S. O. H. Madgwick, A. J. L. Harrison, and R. Vaidyanathan, "Estimation of IMU and MARG orientation using a gradient descent algorithm," *IEEE Int. Conf. Rehabilitation Robotics*, 2011.

[8] K. Shoemake, "Animating rotation with quaternion curves," in *Proc. SIGGRAPH*, vol. 19, no. 3, 1985, pp. 245–254.

[9] A. Poulose, J. Kim, and D. S. Han, "A Sensor Fusion Framework for Indoor Localization Using Smartphone Sensors and Wi-Fi RSSI Measurements," *Applied Sciences*, vol. 9, no. 20, 2019.

[10] B. Wei, M. Gao, F. Li, C. Luo, S. Wang, and J. Zhang, "rWiFiSLAM: Effective WiFi Ranging Based SLAM System in Ambient Environments," *IEEE Robot. Autom. Lett.*, vol. 9, no. 6, pp. 5362–5369, 2024.